Adreno as a native backend in Modular’s compiler

```
mojo build --target-accelerator adreno840 kernel.mojo  →  one Vulkan compute shader (.spv) per kernel
```

Piece	What it does	In the fork
Target	adreno840: Vulkan SPIR-V, 64-lane subgroups, cooperative matrices for the matrix units	SPIRVTraits.cpp
GPU intrinsics	thread ids, barriers, shuffles, warp reductions and math become SPIR-V builtins	SPIRVLowering.cpp
Vulkan kernel ABI	pointers become storage buffers, numbers a params buffer; structured control flow; matrix types	VulkanKernelABI.cpp
SPIR-V writer	LLVM’s SPIR-V backend inside mojo, patched for matrices in Vulkan shaders	SPIRVBackend.cpp, 2 LLVM patches
MAX	CoopMatrix type; matmul picks a matrix-unit kernel on adreno840; graphs compile for vulkan:adreno840	mma_spirv.mojo, matmul/gpu/adreno

Built on Modular’s open-source Mojo compiler · runs on Qualcomm’s stock Vulkan driver

A Mojo kernel, and the SPIR-V it becomes

Subgroup: a lane shuffle and a warp reduction (64 lanes)

```
ss += shuffle_xor(ss, UInt32(8))
var s = warp_sum(x)
```

```
%lane = OpLoad %u32 %SubgroupLocalInvocationId
%src = OpBitwiseXor %u32 %lane %8
%t = OpGroupNonUniformShuffle %f32 %sg %ss %src
%s = OpGroupNonUniformFAdd %f32 %sg Reduce %x
```

Cooperative matrix: one 64 × 64 fp32 tile, from A (64 × 8) and B (8 × 64) in shared memory

```
var acc = Acc.zero()
var a = A.load(sa, 16)
var b = B.load[row_major=False](sb, 16)
acc = a.multiply_add(b, acc)
acc.store(out, ldy)
```

```
%C = OpTypeCooperativeMatrixKHR %f32 %sg 64 64 %Acc
%z = OpConstantComposite %C %f32_0
%a = OpCooperativeMatrixLoadKHR %A %sa %RowMajor %16
%b = OpCooperativeMatrixLoadKHR %B %sb %ColMajor %16
%c = OpCooperativeMatrixMulAddKHR %C %a %b %z
OpCooperativeMatrixStoreKHR %out %c %RowMajor %ldy
```

A, B and Acc are CoopMatrix[DType.float32, …] with MATRIX_A, MATRIX_B and ACCUMULATOR. Every pointer argument becomes storage buffer 0, 1, 2…; every number a slot of %params.

spirv-dis of normalize_qk, gemv and p_gemm_t2 from mojo-adreno’s builds · ids renamed for reading · %sg = subgroup scope

Mojo kernels, run by Mojo host code

Write in Mojo
the kernels and the forward pass, as for any GPU

Compile for Android
arm64 host code, every kernel inside as SPIR-V

Weights on the phone
downloaded from Hugging Face, converted there

Run through MAX
its DeviceContext, on a Vulkan driver plugin

```
var ctx = DeviceContext(api="vulkan")
for l in range(L):
  ctx.enqueue_function[rmsnorm](
    x, w[l], xn, grid_dim=1, block_dim=256)
  self.proj(wqkv[l], xn, qkv)  # gemv
  ...
ctx.enqueue_copy(host, logits)
ctx.synchronize()
```

Model	Forward pass in Mojo	C runner it replaces	Decode, Mojo · C
Llama 3.2 1B	model.mojo, 150 lines	563 lines	24.7 · 23 tok/s
Bonsai 2 2B	the port’s own model.mojo	747 lines	74–76 · 80 tok/s
Qwen3-8B	model.mojo, 165 lines	414 lines	25.1 · 25 tok/s

> A launch through DeviceContext costs 6.5 µs: the plugin batches 64 launches a submission, so decode and batched prefill keep the C runners’ speed.

MAX’s runtime cross-built for Android · libM_driver_vulkan.so: MAX’s driver C API on Vulkan · Galaxy S26 Ultra, stock driver

Prefill and decode on the Adreno 840

Model	Decode, Mojo host code	Decode, C runner	Prefill rate, batched	Prefill, one token at a time	Checked
Llama 3.2 1B, fp16 2.47 GB	**24.7 tok/s** · 40.6 ms	23 tok/s · 43 ms	**684 tok/s** · 0.63 s, 28× · Mojo 0.65 s	24 tok/s · 17.7 s	PyTorch 48/48
Llama 3.2 1B, 4-bit 1.18 GB	**78.1 tok/s** · 12.8 ms	75 tok/s · 13 ms	**695 tok/s** · 0.62 s · Mojo 0.59 s	not measured	PyTorch 48/48
Bonsai 2 2B, ternary 0.48 GB	**74–76 tok/s** · the port's model.mojo	80 tok/s · 12.4 ms	**474 tok/s** · 0.87 s, 6×	80 tok/s · 5.2 s	MLX 16/16
Qwen3-8B, ternary 2.19 GB	**25.1 tok/s** · 39.8 ms	25 tok/s · 40 ms	**103 tok/s** · 3.95 s, 4.4× · 1000 tokens 106 tok/s	23.4 tok/s · 17.4 s	MLX 16/16

Decode: memory bandwidth

A token's time is the bytes of weights read: 4-bit took Llama from 43 to 13 ms; Mojo launches cost nothing measurable. After 1000 prompt tokens, Qwen3's KV cache slows it to 17.0 tok/s.

Prefill: the matrix units

All prompt tokens at once reuse every weight: fp32 cooperative-matrix products at about 1.8 TFLOP/s, 4 to 28 times faster, from Mojo as from C.

Prompts: Llama 431 tokens (prefill) and 46 (decode), Bonsai 412 and 19, Qwen3 408 (Llama's passage) and 19 · Galaxy S26 Ultra, Z Fold (Mojo prefill) and SM8850 reference phone

Next: MAX itself on the Adreno

26/26 MAX test graphs compile for vulkan:adreno840

MAX’s matmul runs on the matrix units: 3/3 pass

MAX’s DeviceContext runs on the phone: its runtime on Android, our Vulkan plugin

Advantages

- One model definition: the same MAX graph as on NVIDIA and AMD, no forward pass per model
- The graph compiler fuses and plans memory: far fewer than today’s 615 to 721 launches a token
- Pipelines bring the tokenizer, sampling, KV cache and serving
- Compile on the Mac, ship a compiled model to the phone

Challenges

- MAX’s graph executor still needs its Android build; the Mojo runtime and device plugin have theirs
- Kernels must fit Adreno: 64-row tiles, fp16-only sums, 32 KB shared memory
- Ternary and 4-bit formats are custom kernels today
- Driver builds differ: every compiled graph needs checking on the device
- Phone memory is shared and small: weights mapped once, compiling off the device

MAX’s device plugin interface: about 60 C functions, now on Vulkan (mojo-adreno driver/) · runtime: --config=android-arm64